GLA UNIVERSITY, MATHURA

Department of Computer Engineering Application



Topic: MINI PROJECT SYNOPSIS ON

CIVICCONNECT - MUNICIPAL COMPLAINT PORTAL (MERN STACK)

Submitted by:

1) Pradeep Kumar
University Roll No:-2415001124

2) Piyush Gautam
University Roll No:-2415001109

3) Anushka Bansal
University Roll No:-2415000304

4) Pratiksha Sharma
University Roll No:-2415001171

5) Krish Agrawal
University Roll No:-Not Provided

Submitted to:

Mr: Akash Kumar Choudhary
Master Trainer

DECLARATION

We hereby declare that the mini project titled **"CivicConnect — Municipal Complaint Portal (MERN Stack)"** is proposed and will be developed by the undersigned team members as part of our academic mini project. The project will be designed and implemented using the MERN stack and will follow clean coding practices, REST API principles, and Git/GitHub version control.

We further declare that the work presented in this synopsis is intended for academic purposes and that the team members will contribute to the analysis, design, development, testing, documentation, and presentation of the proposed system under the guidance of the assigned mentor.

S.No.	Name of Student	University Roll No.
1	Pradeep Kumar	2415001124
2	Piyush Gautam	2415001109
3	Krish Agrawal	Not Provided
4	Anushka Bansal	2415000304
5	Pratiksha Sharma	2415001171

Mentor: Mr. Akash Kumar Choudhary, Technical Trainer

INDEX

This index follows the organization of the supplied synopsis format and maps the sections of the CivicConnect project.

S. NO.	TOPIC	PAGE
1	Introduction	6
2	System Requirements	9
3	Hardware Requirements	9
4	Front End and Back End	9
5	Idea / Primary Reason to Choose This Project	7
6	Objective	7
7	Module Description	10
8	Availability / Scope	8
9	DFD — 0 Level, 1 Level and 2 Level	10
10	Bibliography / References	11–13

Project Title: CivicConnect — Municipal Complaint Portal (MERN Stack)

Technology: MongoDB, Express.js, React.js, Node.js, REST APIs, Git/GitHub

INTRODUCTION

In today's rapidly developing cities, citizens face various civic problems such as garbage accumulation, damaged roads, non-functional streetlights, water supply issues, drainage problems, and other municipal service-related concerns. Reporting these issues through traditional methods can be time-consuming and often provides limited information about the progress of a complaint. Citizens may also face difficulties in communicating their problems directly to the concerned authorities.

To address these challenges, **CivicConnect** provides a digital platform that connects citizens with municipal authorities. It enables citizens to report their civic issues online and provides a systematic way to manage and track complaints. The system focuses on making the complaint process more convenient, transparent, and organized.

About the Project

CivicConnect is a web-based **Municipal Complaint Portal** developed using the **MERN Stack**, consisting of MongoDB, Express.js, React.js, and Node.js. The main purpose of the project is to digitalize the process of reporting and managing civic complaints.

Citizens can register on the platform and submit complaints by selecting the appropriate category and providing details such as description and location of the issue. Municipal authorities can access these complaints through the system, review the reported problems, update their status, and manage them according to their progress. This allows citizens to track their complaints and get better visibility into the resolution process.

The system uses **React.js** to provide an interactive user interface, while **Node.js and Express.js** handle backend operations and REST API communication. **MongoDB** is used for storing user and complaint-related data. Git and GitHub can be used for version control and collaborative development.

Abstract

The **CivicConnect** project aims to provide an efficient and transparent solution for municipal complaint management. The system allows citizens to report civic problems through an online platform instead of relying on traditional manual methods. It also provides municipal authorities with a centralized system to receive, organize, update, and track complaints.

By providing features for complaint registration, categorization, status updates, and tracking, the system helps improve communication between citizens and authorities. The use of modern web technologies makes the platform scalable, user-friendly, and suitable for managing different types of civic issues. Overall, CivicConnect aims to reduce manual effort, improve transparency, and support faster and more organized resolution of municipal complaints.

Keywords

CivicConnect, Municipal Complaint Portal, Complaint Management, Citizen Services, MERN Stack, MongoDB, Express.js, React.js, Node.js, REST API, E-Governance, Digital Governance.

PRIMARY REASON TO CHOOSE THIS PROJECT

Civic problems are common in urban and semi-urban areas, but citizens may not always have a convenient way to report them and know what happens after a complaint is submitted. A digital portal can organize these complaints, reduce information gaps, and make the progress of each issue easier to follow.

THE MAIN OBJECTIVE OF THE PROJECT

1. To provide citizens with an easy online platform for reporting civic issues.

2. To allow users to view and track the current status of submitted complaints.

3. To provide municipal staff with a centralized interface for managing complaints.

4. To categorize complaints such as potholes, garbage, streetlights, drainage, and other civic issues.

5. To maintain complaint and user information securely in a MongoDB database.

6. To develop REST APIs using Node.js and Express.js for communication between frontend and backend.

7. To build a responsive and understandable interface using React.js.

8. To use Git/GitHub for version control and collaborative development.

Expected Academic Outcome: The project will help the team apply concepts of full-stack development, database management, API design, software testing, clean coding, and collaborative version control in one practical application.

SCOPE OF THE PROJECT

The initial scope covers citizen registration/login, complaint submission, complaint categorization, complaint tracking, and an administrative workflow for reviewing and updating complaint status. The portal can be accessed through a web browser on supported devices.

Possible future extensions include location-based complaint mapping, image attachments, notifications, analytics dashboards, department-wise assignment, escalation rules, and integration with municipal service systems.

WORKING METHODOLOGY OF THE PROJECT

1. Requirement Analysis: Identify users, civic complaint types, required information, and expected workflows.

2. UI/UX Design: Design simple React pages for citizens and municipal staff.

3. Backend Development: Create Node.js/Express.js REST APIs for authentication, complaints, status updates, and related operations.

4. Database Integration: Design MongoDB collections for users, complaints, categories, and status history.

5. Integration: Connect the React frontend with backend REST APIs.

6. Testing: Test forms, API responses, validation, status updates, and common error cases.

7. Version Control: Use Git/GitHub for commits, collaboration, and maintaining project versions.

DETAILS ABOUT THE HARDWARE AND THE SOFTWARE

SYSTEM REQUIREMENTS

Supported Operating System: Windows 10/11, Ubuntu/Linux, or another OS capable of running Node.js and MongoDB.

Software Required: Visual Studio Code, Node.js and npm, MongoDB/MongoDB Atlas, Git, GitHub account, and a modern web browser such as Chrome or Edge.

HARDWARE REQUIREMENTS

Processor: Dual-core processor or better.

RAM: Minimum 4 GB; 8 GB or more is recommended for comfortable development.

Storage: At least 5 GB of free space for project files, dependencies, and development tools.

Network: Internet connection for package installation, GitHub, and cloud database access if MongoDB Atlas is used.

LISTING OUT TESTING TECHNOLOGY

The project can be tested using browser-based functional testing, API testing with Postman, form validation testing, and integration testing of frontend, backend, and database operations.

FRONTEND AND BACKEND

Frontend: React.js, HTML, CSS, and React-based components for user interaction and responsive pages.

Backend: Node.js with Express.js, REST APIs, request validation, business logic, and database communication.

MODULE DESCRIPTION

1. Citizen Module: Registration/login, submit complaint, select issue category, add description, and view complaint status.

2. Complaint Management Module: Stores complaint details, generates a complaint record, and maintains status such as Submitted, In Progress, Resolved, or Closed.

3. Municipal/Admin Module: Authorized staff can review complaints, update status, assign or process issues, and maintain complaint records.

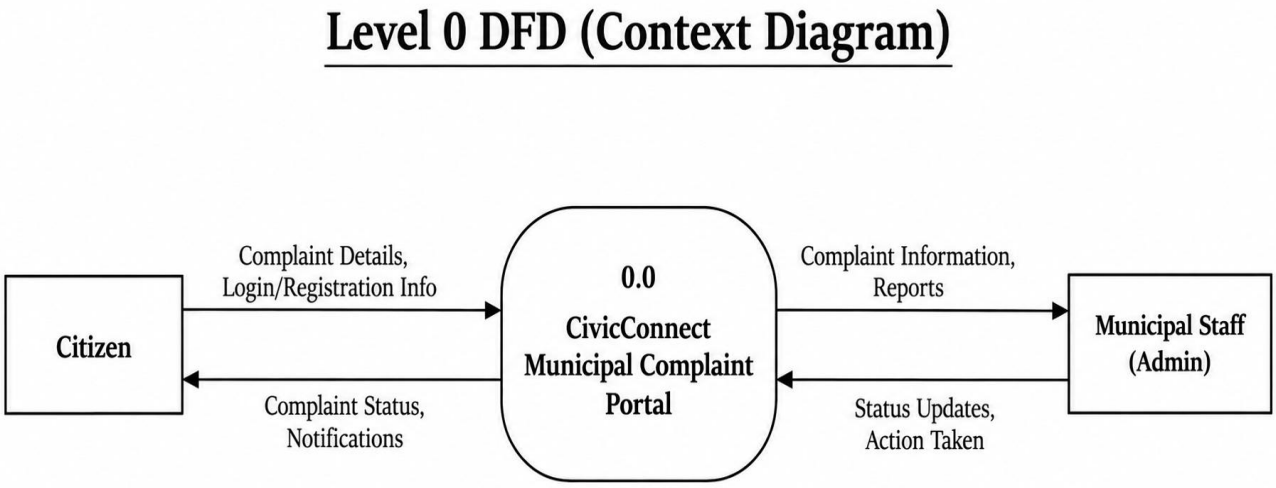
4. Database Module: MongoDB stores user profiles, complaints, categories, status information, and other required data.

5. API Module: Express.js REST endpoints provide communication between the React frontend and MongoDB-backed server.

DATA FLOW DIAGRAMS

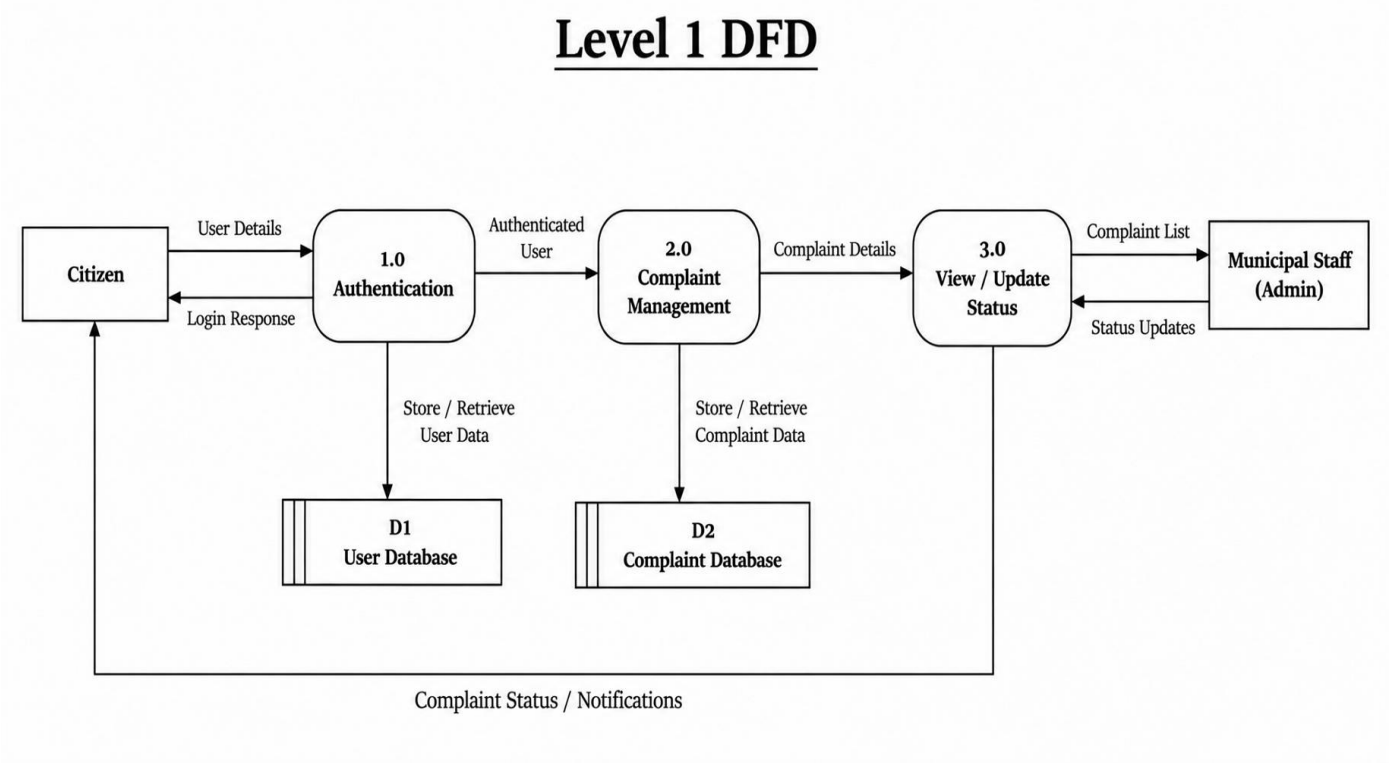
A data flow diagram (DFD) illustrates how data is processed by the system in terms of inputs, outputs, processing, and storage.

Level 0 DFD:

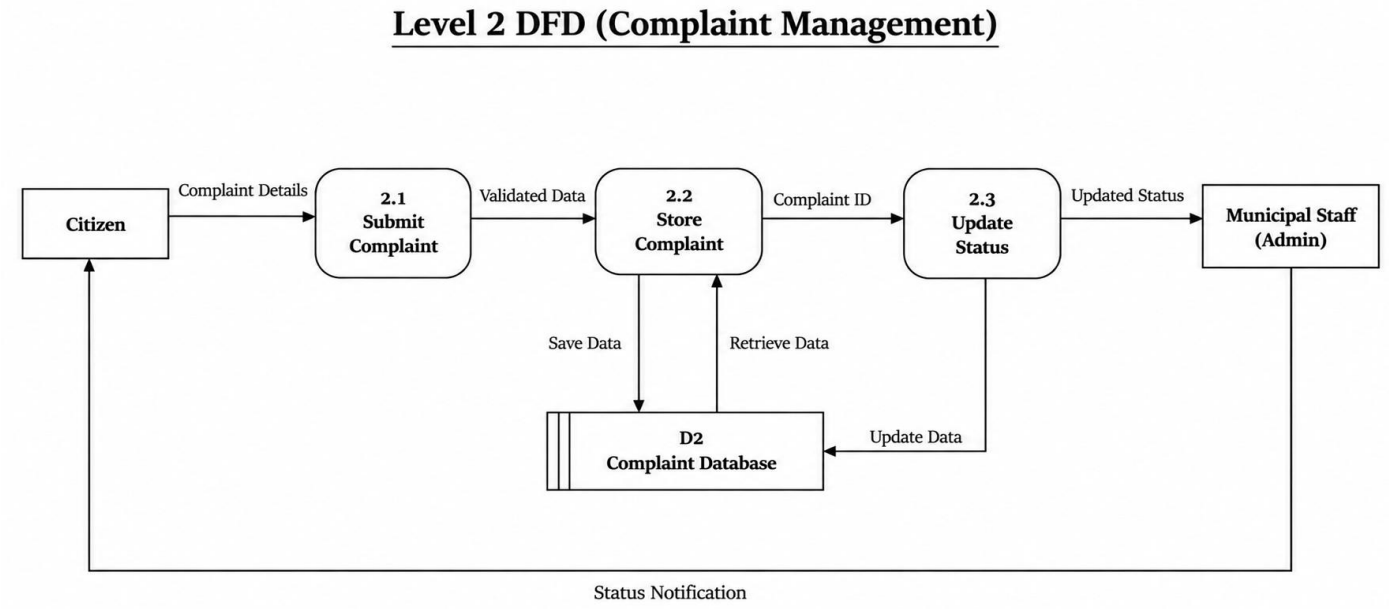


MODULE DESCRIPTION

Level 1 DFD:



Level 2 DFD:



REFERENCES

[1] MongoDB Documentation, “MongoDB Manual,” MongoDB, Inc.

[2] Express.js Documentation, “Express — Node.js web application framework.”

[3] React Documentation, “React — The library for web and native user interfaces.”

[4] Node.js Documentation, “Node.js Documentation.”

[5] Git Documentation, “Git Reference Manual.”

[6] GitHub Documentation, “GitHub Docs.”

BIBLIOGRAPHY

The above documentation sources are intended to support implementation and understanding of the MERN stack, REST API development, database operations, and Git/GitHub version control used in CivicConnect.

AVAILABILITY AND WORKFLOW

Availability:

CivicConnect is proposed as a browser-based web application. A citizen can use the portal from a supported device with a web browser and network access. Municipal staff can access the administrative functions through authorized accounts.

Basic Workflow:

1. Citizen opens CivicConnect.

2. Citizen registers or logs in.

3. Citizen submits a civic complaint with category and description.

4. The backend validates the request and stores the complaint in MongoDB.

5. Municipal staff views the complaint through the admin interface.

6. Staff updates the complaint status as work progresses.

7. Citizen checks the portal to track the latest status.

8. Resolved complaints are marked completed/closed and remain available as records.

PROPOSED STATUS FLOW

Submitted → Under Review → In Progress → Resolved → Closed

CONCLUSION

CivicConnect proposes a practical solution for managing municipal complaints through a centralized web portal. It brings citizens and municipal staff onto one platform where complaints can be submitted, stored, processed, and tracked in an organized manner.

The MERN stack provides a suitable full-stack foundation for this application: React.js supports an interactive frontend, Node.js and Express.js support REST-based backend services, and MongoDB provides flexible data storage. Git/GitHub will support version control and collaborative development.

The project is expected to improve transparency in complaint handling and provide a clear digital workflow for civic issue reporting and resolution tracking. The design can later be extended with maps, notifications, analytics, attachments, and more advanced municipal integrations.

FUTURE SCOPE

- GPS/map-based complaint location and ward identification.
- Email/SMS or in-app notifications for status changes.
- Photo/video attachment support for better issue verification.
- Analytics dashboard for frequently reported civic issues.
- Department-wise assignment and escalation mechanisms.

TEAM

S.No.	Name of Student	University Roll No.
1	Pradeep Kumar	2415001124
2	Piyush Gautam	2415001109
3	Krish Agrawal	Not Provided
4	Anushka Bansal	2415000304
5	Pratiksha Sharma	2415001171